— PAYMENTS & RETRIES

Tap Pay twice, get charged once

00 The problem

Retries are normal: timeouts, lost replies, double taps. Without protection, every retry can become a **second payment**.

01 Idempotency key

- The client creates **one unique ID per payment** (e.g. a UUID), not per request.
- Every retry sends the **same** key, often as a header: `Idempotency-Key: pay_8f3a21`

02 The server check

Key not seen
Process the payment. Save key → result.

Key seen
Return the saved result. No new debit.

03 Two retries at the same moment

Insert the key atomically (unique constraint or a lock). The first request wins. The second gets "in progress" (often HTTP 409) and retries later.

04 Real world

Stripe's API accepts an `Idempotency-Key` header on POST requests for exactly this reason.

— INTERVIEW READY

Details that impress interviewers

01 Which HTTP methods are idempotent?

- **GET, PUT, DELETE** are idempotent by definition.
- **POST** is not. That's why payment APIs need keys.

02 Storing the keys

- Keep `key → status, response, request hash` in a DB or Redis.
- Expire keys after a set window, often 24 hours.
- Same key with a **different** request body? Reject it as an error.

03 Save it with the payment

Write the result in the same transaction as the debit. Otherwise a crash can leave money moved but no saved result.

04 Say it in 20 seconds

"The client sends a unique key with every retry. The server stores key and result atomically. A repeat key returns the saved result, so the payment happens once."

05 Common mistakes

- Making a **new key on every retry**.
- Check-then-insert without atomicity (race condition).
- Not storing the response, so retries can't get it back.